$\mathcal{M}odel\mathcal{X}$: A Novel Transfer Learning Approach Across Heterogeneous Datasets

***Abstract*—Leveraging an existing performance model to predict the runtime of a new application on a new system can save days and weeks of data collection time. However, knowledge transfer between High Performance Computing (HPC) systems can be challenging due to data heterogeneity caused by differences in data collection methods, architectural or application-specific individuality. This results in (1) sets of performance features that have significantly different names, orders, or the number of performance features that do not match between two datasets (*heterogeneous domains)*, or (2) distribution shifts between datasets although their feature names match (*homogeneous domains)*. While existing transfer learning techniques can handle mild distribution shifts, they fail to transfer knowledge when the source and target features do not match. This work introduces a novel transfer learning methodology–`Cross Prediction Model` ($\mathcal{M}odel\mathcal{X}$), which overcomes the large distribution discrepancy between *homogeneous domains* and enables transfer learning between *heterogeneous domains*. Extensive evaluations show that $\mathcal{M}odel\mathcal{X}$ outperforms traditional transfer learning methods for all experiments using 11 HPC and 4 Machine Learning (ML) datasets. To the best of our knowledge, this is the first methodology to enable knowledge transfer between two heterogeneous domains with no matching features. Finally, we demonstrate an application of $\mathcal{M}odel\mathcal{X}$ to an HPC job scheduling scenario using real-world job traces where it helps to reduce the job turnaround time of a set of jobs by 71%.**

***Index Terms*—Transfer learning, Heterogeneous domains, Predictive performance, Intelligent scheduling**

I. Introduction

Performance modeling in HPC is essential for understanding how applications interact with hardware, identifying bottlenecks, and optimizing resource allocation. These models can guide users to configure jobs, job schedulers to make optimal resource allocation decisions, and inform procurement decisions to ensure that hardware investments align with application requirements. Prior studies have shown that improper resource configuration can lead to performance slowdowns of up to $4\times$ [1], highlighting the need for accurate prediction.

However, constructing performance models requires extensive data collection for every new system and application, which can take days or weeks. To mitigate this, Transfer Learning (TL) [2]–[5], an ML technique, has been applied in HPC to reuse knowledge across applications and architectures, reducing the burden of collecting new training data. This work investigates two common knowledge transfer scenarios in HPC: (a) Cross-architecture transfer (**xArch**): Can knowledge from an application running on one system be transferred to predict its performance on a different system? (b) Cross-application transfer (**xApp**): Can knowledge from multiple applications on a given system be leveraged to predict the performance of a new application on the same system?

While TL has been effective in handling distribution shifts–where input features remain the same but their statistical distributions change–it fundamentally assumes that source and target datasets have identical feature names, counts, and orders, or at least a strict subset relationship. However, in HPC, these assumptions do not always hold due to *feature heterogeneity*–differences in feature names, counts, and orders across datasets–which stems from variations in data collection tools, application configurations, and system-specific performance counters. Consequently, existing TL methods fail to use new samples from the target domains to adapt or generalize an existing source performance model unless a model is retrained from scratch using new samples.

Table I illustrates common job-, application-, hardware-, and system-level features used in performance models [1], [5], [6]. Some of the high-level features, such as task count, thread count, and network bandwidth, are relatively easy to standardize across datasets and have been successfully used in prior work for xApp transfer [1], [5]. However, these features are insufficient for xArch transfer because execution characteristics depend heavily on architecture-specific factors, such as cache hierarchies, NUMA layouts, and memory bandwidth. Hence, to accurately model performance across architectures (xArch), hardware counters should be considered as features.

However, in the xArch scenario, including these counters leads to feature mismatches, as counter names vary across architectures, and some counters may be missing entirely. For example, L3-cache counters present on Intel architectures do not exist on IBM BG/Q (BGQ) systems. Existing TL methods, such as fine-tuning [7] and linear probing [8], assume either (1) identical features or (2) a strict subset relationship, neither of which holds in xArch scenarios. Consequently, models trained on one system fail to generalize when new, unseen features appear in the target dataset.

To address these challenges, we introduce a novel few-shot learning-based approach (Section III) that enables knowledge transfer across both xApp and xArch scenarios in a data-efficient manner. Specifically, it handles (1) Feature mismatch in heterogeneous datasets without manual intervention, and (2) Severe distribution shift in homogeneous datasets. Instead of training a model from scratch every time new data is seen, our approach adapts any pre-trained source model at test time using only a few labeled target samples (data efficient), which

reduces data collection time from days to seconds. Unlike most TL methods, this work builds a regression model for performance prediction.

We hypothesize that our proposed two-phase modeling strategy–$\mathcal{M}odel\mathcal{X}$–enables knowledge transfer across heterogeneous datasets by jointly learning (1) feature alignments that adapt target features into the source model's learned space and (2) residual relationships that account for unaligned variations. The input alignment phase (`Input Alignment Model` ($M_{\mathcal{A}}$)) handles moderate distribution shifts. If unaligned target features remain, the second phase (`Stacked Input Alignment Model` ($M_{\mathcal{R}}$)) stacks a neural network, initialized by $M_{\mathcal{A}}$, that trains on the few-shot samples to account for both aligned and unaligned features. It combines these learned relationships with the source model's predictions, capturing unique patterns missed during alignment, correcting bias, and improving generalization. If $M_{\mathcal{A}}$ effectively aligns the target features in the first place, then the additional stacked model becomes unnecessary. This is handled by $\mathcal{M}odel\mathcal{X}$ seamlessly such that it picks the best model based on their prediction errors on the few-shot samples.

This work assumes that knowledge transfer is possible for xApp and xArch scenarios because, despite differences in feature names, counts, and distributions, there exists shared high-level performance characteristics across similar classes of applications. This transferability stems from fundamental relationships between system resources and application behavior, which persist across different programs within the same performance category. For instance, the performance of compute-bound kernels degrades at lower CPU/GPU frequency, whereas communication-bound kernels remain largely unaffected; network bandwidth strongly influences communication-bound workloads but has little impact on compute-bound applications. However, in cases where no meaningful relationships exist, transferability is inherently limited.

In Section III-G, we propose an explainability metric–`severity`–to quantify the difficulty of knowledge transfer and use it to explain when input alignment alone is sufficient or when residual modeling is necessary. Our extensive evaluation across 800 experiments spanning 32 source-target pairs demonstrates that $\mathcal{M}odel\mathcal{X}$ consistently outperforms all existing transfer learning methods and matches a fully supervised model trained with $100\times$ more target-domain samples with only 1%–5% of the data, eliminating the need for costly data collection and model retraining.

To summarize our contributions, we:

- Design a novel data-efficient **few-shot transfer learning method for structurally heterogeneous data** that enables performance modeling across datasets with completely different feature sets.
- Develop a `severity` measure to quantify the difficulty of knowledge transfer.
- Evaluate on an extensive set of datasets in both HPC and ML demonstrating that $\mathcal{M}odel\mathcal{X}$ consistently outperforms existing transfer learning methods.
- Demonstrate the impact of $\mathcal{M}odel\mathcal{X}$ on job scheduling by reducing job turnaround time by 71% and amount of resource required to achieve the same turnaround time by 42%, outperforming existing scheduling heuristics [9], [10].

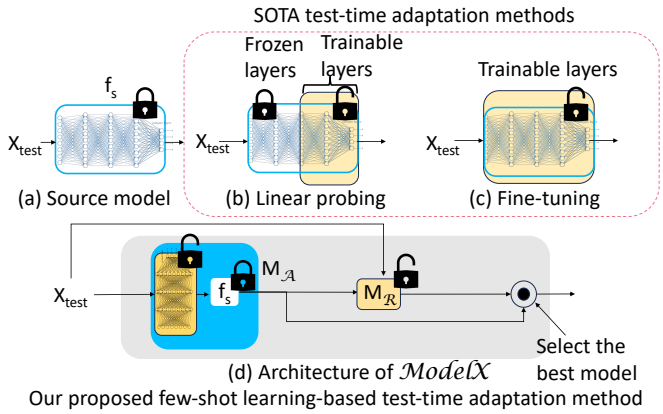


Fig. 1: Block diagrams of different state-of-the-art (SOTA) transfer learning approaches. A closed lock symbol denotes frozen layers, and an open lock symbol denotes trainable layers. The $\odot$ denotes a comparison operator.

To assess the generalizability of our method, in Section V-C, we also evaluate our approach with several ML benchmark datasets. While a sophisticated source model could enhance baseline performance, $\mathcal{M}odel\mathcal{X}$ can be applied independent of its architecture and how it is built, making source model design a complementary problem.

The rest of the paper is structured as follows: Section II outlines the baseline transfer learning methods used for comparison; Section III details the design of $\mathcal{M}odel\mathcal{X}$ and `severity`, covering source model construction, input alignment, residual modeling, and application of $\mathcal{M}odel\mathcal{X}$; Section IV provides an overview of the evaluation setup; Section V evaluates $\mathcal{M}odel\mathcal{X}$ across five different scenarios, demonstrating its ability to generalize source model and impact on job scheduling. Finally, Section VI reviews prior work on transfer learning, and Section VII concludes the paper.

II. Background

This section defines key terminologies, including transfer learning and test-time adaptation, from a performance modeling perspective. Figure 1 illustrates these concepts, where 1(a) shows a source model and 1(b) and 1(c) depict common transfer learning strategies from the literature. In Section V, we evaluate the effectiveness of our proposed test-time adaptation methods against these baselines.

No test-time adaptation: Figure 1(a) represents the scenario where an existing source model f_s is used without modification (hence the lock symbol) during test time.

Linear Probing (LP) [11]: Figure 1(b) describes a traditional transfer learning approach called Linear Probing (LP) where

TABLE I: Subset of feature names, where N represents the number of samples and p represents the number of features. To conserve space, we use regular expressions to represent hardware counters in the Intel Sandy Bridge (SB) and BGQ datasets and highlight the performance target variables in bold.

App.	Example Features
HPC Datasets	
`CG`, `LU`, `FT`, `Kripke1`, `CoMD`, N=3180, p=7	Power Cap, Task Count, Core Count, Placement, and Bandwidth, **runtime**
`Kripke2`, N=17386, p=23	DRAMPowerPerNode, ProcessorPowerPerNode, Ranks, App. specific input parameters, OMP, PKG_LIMIT, DRAM_LIMIT, AvgInst, AvgIpc, AvgArithFpu, AvgFreq, AvgTemp, ProcessorPower, DRAMPower, Nesting Order, **ExecTime**
`Hypre`, N=50396, p=21	DRAMPowerPerNode, ProcessorPowerPerNode, Ranks, OMP, PMX, NS, MU, AvgIPC, Smoother, AvgTSC, AvgTemp, ProcessorPower, DRAMPower, Solver-related parameters, **ExecTime**
XSBench and OpenMC on SB, N=200, p=121	NumThread, InputSize, **EfficiencyLoss**, perf::[MEM|DTLB|LLC]_[MISS|STALL], perf::[L_1|L_2|L_3|]_[MISS|STALL]
XSBench and OpenMC on BGQ, N=200, p=145	NumThread, Input-Size, **EfficiencyLoss**, PEVT_[XU|AXU|L1P|STL]_[MISS|], PAPI_[BR|STL|SYC])_[STALL|CYC|MISS]
ML Datasets	
Airfoil, N=1503, p=5	Frequency, Angle of attack, Chord length, Free-stream velocity, Suction side, **Scaled sound pressure level**
NO2, N=500, p=8	**NO2**, Cars per hour, temperature, wind speed, temperature difference, wind direction, hour of day, day number
Crime, N=1994, p=127	population, householdsize, PctEmploy, PctIlleg, medIncome, perCapInc, PctPopUnderPov, **ViolentCrimesPerPop**
SkillCraft, N=3395, p=16	GameID, APM, SelectByHotkeys, AssignToHotkeys, MinimapRightClicks, NumberOfPACs, **ActionLatency**

only the output layer of the source model is adapted with a few test samples from the target domain. This test-time adaptation method utilizes the source model's pre-learned embeddings without modification and enhances them with new knowledge from the test domain. However, LP does not work if the number of features between the test samples and f_s differs.

Fine-tuning [7], [12]: Figure 1(c) describes the Fine-tuning (FT) method, a special case of LP where the weights of all of the layers of the source model are updated. The main difference between LP and FT is that LP retains weights of some of the layers (known as frozen layers), while FT updates all of the layers. Similar to LP, fine-tuning does not work if the number of features between the test samples and f_s differs.

Retrain with few-shot: This work builds a model from scratch using a few labeled samples from the target domain (not TL).

Supervised-Target: As an upper-bound baseline, we train a fully supervised model on a large target-domain dataset for each experiment. This represents the best possible performance without transfer learning.

III. Our Approach

We propose a new TL method—$\mathcal{M}odel\mathcal{X}$ inspired by Neural Machine Translation [13], [14] in Natural Language Processing (NLP). Figure 1 (d) illustrates the architecture of our proposed method. Specifically, our approach addresses the issues that currently impede effective knowledge transfer across homogeneous and heterogeneous domains due to (1) distribution shifts, (2) unknown feature ordering, and (3) uncommon features between source and target domains. The architecture of $\mathcal{M}odel\mathcal{X}$ includes two steps: (1) Input Alignment ($M_{\mathcal{A}}$): Marked in the orange-patterned area in Figure 1(d) and (2) Residual Model marked as $M_{\mathcal{R}}$ in Figure 1(d). $M_{\mathcal{A}}$ aligns the features of the target domain with the source model by selectively mapping the target features on parts of the relevant source features, thereby handling feature name, number, and order mismatch. To handle target features with no direct equivalent, $M_{\mathcal{R}}$ stacks an additional neural network, which learns the residual relationships from few-shot target samples and initial predictions from the source model.

Table I presents examples of features in the performance datasets that are from various layers of the computing ecosystem, such as algorithm input, execution parameters, and hardware performance counters. The labels in these datasets are performance metrics, such as runtime, speedup, and parallel efficiency, that are collected after the application is executed.

III-A Datasets

We define the source dataset as $D_s = \{X_s, Y_s\}$, where $X_s \in \mathbb{R}^p$ represents the features, $Y_s \in \mathbb{R}^1$ represents the labels, and $|D_s| = N_s$. Each sample $x_i \in X_s$ is a vector of p features, such as hardware performance counters or configuration parameters, while $y_i \in Y_s$ is the corresponding target label such as execution time, power usage, or parallel efficiency loss. Since this work primarily contributes a methodology, its applicability is not limited by the choice of features or target metrics, as these are determined by a user's analysis goals.

Similarly, the target dataset is defined as $D_t = D_t^L \cup D_t^U$, where $D_t^L = \{X_t^L, Y_t^L\}$, $|D_t^L| = N_t^L$ is the number of *labeled few-shot samples* with $X_t^L \in \mathbb{R}^q$ as the features and $Y_t^L \in \mathbb{R}^1$ as the target performance metric, and $D_t^U = \{X_t^U\}$ contains N_t^U unlabeled samples from the target domain with $X_t^U \in \mathbb{R}^q$.

III-B Preprocessing

In this work, the parameters in X_s and X_t include both discrete and continuous variables. To handle these uniformly, discrete variables are transformed into multiple binary variables using a technique called one-hot encoding, where each binary variable represents a discrete state. Continuous parameters are scaled to zero mean and unit variance through centering and standardization, which is critical for ML estimators to perform accurately. For datasets containing more complex feature types, such as graphs or trees, this preprocessing module needs to be extended to create representations and combined with the representation of the rest of the datasets. Further research is needed to design such aggregation techniques, which is complementary to the problem studied in this work.

III-C Source Model Building

In this work, we select deep neural networks (DNNs) as the architecture to train as a source model due to their modularity and adaptability. Unlike tree-based methods, we can train a subset of layers while keeping others frozen (needed for test-

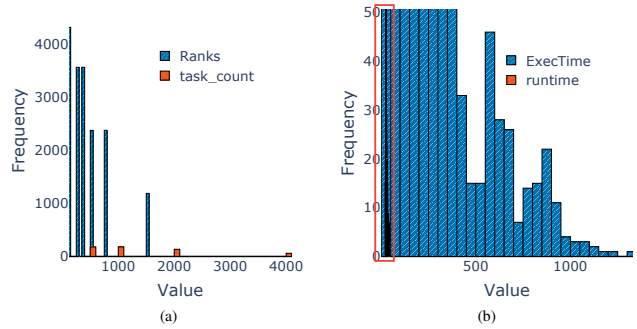


Fig. 2: Two examples of distribution shifts. (a) The feature `Ranks` in one dataset maps to `task_count` in another, capturing the same concept but exhibiting different value distributions. Manually aligning them requires domain expertise, is data-dependent, and does not scale. (b) Prediction metrics can also shift. $\mathcal{M}odel\mathcal{X}$ addresses this challenge by automatically aligning features and their values without manual effort.

time adaptation). Their modular structure also allows us to stack multiple DNN models. In the future, other advanced DNN-based architectures, such as transformers and generative models, can be integrated without altering our approach.

The source model learns a function f_s, as defined in Equation 1, to predict $\hat{Y}_s$ from X_s. The difference between $\hat{Y}_s$ and the true target labels, Y_s, are used to calculate the loss, L_s (Equation 2). The complexity of the source model depends on the dataset. For experiments, we use the automated hyperparameter tuning framework Optuna [15] to identify the number of layers and neurons needed for the source models.

$$f_s(X_s) \rightarrow \hat{Y}_s, X_s \in R^{p} \tag{1}$$

Number of source features

Source model's predictions

$$L_s = \sum_{i=1}^{N_s} (Y_s - \hat{Y}_s)^2 \tag{2}$$

True target labels

This work focuses on test-time adaptation to enhance the generalizability of any source model, leaving the challenge of building a unified source model from diverse datasets as future work. Training on multiple heterogeneous datasets introduces feature and distribution inconsistencies, making it a complex research problem. While such an approach could improve baseline performance, it is complementary to our goal of isolating and evaluating the benefit of test-time adaptation.

III-D Input Alignment, $\mathrm{M}_{\mathcal{A}}$

Input alignment is a widely used technique in Neural Machine Translation (NMT) to establish correspondences between source and target domains' features. For instance, when using NMT for translation, the subject-verb-object order in one language may differ from other languages. Input alignment helps the downstream task understand and reorder words to produce grammatically correct translations. Similarly, distribution shifts can occur between source and target features in HPC due to differences in feature value distributions, label value distributions, or both. Moreover, due to variations in data collection methods, feature names and orders may differ. Figure 2 shows an example of how *Ranks* in `Kripke2` means the same thing as *task_count* in `Kripke1`; however, they carry a significant distribution shift.

> We propose an input alignment approach for transferring knowledge in HPC where feature names, or orders don't match, or there is a distribution shift in feature values. The approach uses a feed-forward network to map target features to relevant parts of source features.

Mathematically, the alignment operation shown in Equation 3 can be described as learning a non-linear transformation, $\mathrm{M}_{\mathcal{A}}$, that maps the target's feature space X_t^L to the relevant parts of the source to produce an aligned feature space Z_t^L.

Aligned target features

Number of target features

$$\mathrm{M}_{\mathcal{A}}(X_t^L) \rightarrow Z_t^L, X_t^L \in R^{q}, Z_t^L \in R^{p} \tag{3}$$

Features of the target few-shot samples

Number of source features

The input alignment model, $\mathrm{M}_{\mathcal{A}}$ in Figure 1(d) is implemented using a fully connected Neural Network (NN) that is trained through back-propagation [14] by minimizing the loss from the source model's predictions $\hat{Y}_t^L$ on the aligned target features Z_t^L and the corresponding true target labels, Y_t^L for the few-shot samples, as shown in Equation 4 and 5.

Source model

Aligned target features

Features of the target few-shot samples

$$f_s(Z_t^L) \rightarrow \hat{Y}_t^L \equiv f_s(\mathrm{M}_{\mathcal{A}}(X_t^L)) \rightarrow \hat{Y}_t^L \tag{4}$$

Input transformation model

Source model's predictions on aligned target features from few-shot samples

$$L_{\mathrm{M}_{\mathcal{A}}} = \sum_{i=1}^{N_t} (Y_t^L - \hat{Y}_t^L)^2 \tag{5}$$

True target labels of the few-shot samples

Input alignment is necessary for handling distribution shift and heterogeneous domains but not always sufficient: The input alignment layer reduces distribution shift among similar features even though they may have different names or orders. Also, if $p \subset q$, $p \neq q$ and $q > p$, then $\mathrm{M}_{\mathcal{A}}$ projects the q-dimensional target feature space onto the p-dimensional source feature space, aligning the two domains. While this step mitigates structural differences between the feature spaces, it does not account for the residual relationships involving the unaligned features, as these were not part of the source model's training. Consequently, the source model cannot capture these new relationships, necessitating the stacking of an additional model, as described in Section III-E.

III-E Stacking an NN-model after Input Alignment, $\mathrm{M}_{\mathcal{R}}$

To address the residual features in the target domain that cannot be aligned with the source domain, we propose stacking an additional neural network (NN) after the input alignment step. This additional model, referred to as the residual model $\mathrm{M}_{\mathcal{R}}$, learns a function, f_R, to capture the residual correlations. It also uses the $\mathrm{M}_{\mathcal{A}}$ model's predictions $(\hat{Y}_t^L)$ as an additional input. The inclusion of $\hat{Y}_t^L$ ensures that the residual model builds upon the knowledge transferred from the source model rather than discarding it. This design allows the residual model to selectively correct the errors in the $\mathrm{M}_{\mathcal{A}}$ model's predictions while simultaneously leveraging the residual features to improve prediction accuracy. The rationale for concatenating the $\mathrm{M}_{\mathcal{A}}$ model's predictions as input to X_t^L before passing onto the $\mathrm{M}_{\mathcal{R}}$ model's input is that such model stacking [16] concept is effective in generalization, as each model can contribute complementary insights. Here, the $\mathrm{M}_{\mathcal{A}}$ model acts as a baseline predictor, providing a starting point for the residual model to refine its understanding of the target.

The residual model $M_{\mathcal{R}}$ learns a mapping from $(X_t^L, \hat{Y}_t^L)$ to the true target labels (Y_t^L), as expressed in Equation 6. By minimizing the loss function in Equation 7, f_R optimizes the predictions $\hat{Y}_t^{L'}$ to reduce the discrepancy with the true target labels. This step ensures that the final predictions incorporate both aligned features and the residual target-specific information. In cases where the source domain contains more features than the target domain, only the aligned features contribute to knowledge transfer as the residual model is agnostic to the excess features in the source domain.

$$\begin{aligned} \underbrace{\mathrm{M}_{\mathcal{R}}}_{\text{Residual Model}} &: f_R(X_t^L, \hat{Y}_t^L) \rightarrow \hat{Y}_t^{L'} \\ &\equiv f_R(X_t^L, f_s(\mathrm{M}_{\mathcal{A}}(X_t^L))) \rightarrow \hat{Y}_t^{L'} \end{aligned} \tag{6}$$

$$L_{\mathrm{M}_{\mathcal{R}}} = \sum_{i=1}^{N'} (\underbrace{Y_t^L}_{\text{True target labels}} - \underbrace{\hat{Y}_t^{L'}}_{\text{Residual model's predictions}})^2 \tag{7}$$

Although the stacked model $(\mathrm{M}_{\mathcal{R}})$ can capture residual relationships, it may risk overfitting if there is limited domain dissimilarity or underfitting if insufficient few-shot samples are available. To address this, $\mathcal{M}odel\mathcal{X}$ adopts a dynamic strategy, selecting between $\mathrm{M}_{\mathcal{A}}$ and $\mathrm{M}_{\mathcal{R}}$ based on their validation losses during test-time adaptation. The $\odot$ operator, as defined in Equation 8, compares the losses of the two approaches and chooses the one with the minimal validation error for prediction. The $\odot$ operator in Equation 8 can be extended to take multiple inputs from various architectures.

$$\underbrace{\odot}_{\text{Minimize validation loss}} = min(\underbrace{L_{\mathrm{M}_{\mathcal{A}}}}_{\text{Loss from M}_{\mathcal{A}}}, \underbrace{L_{\mathrm{M}_{\mathcal{R}}}}_{\text{Loss from M}_{\mathcal{R}}}) \ ? \ M_{\mathcal{A}} : M_{\mathcal{R}} \tag{8}$$

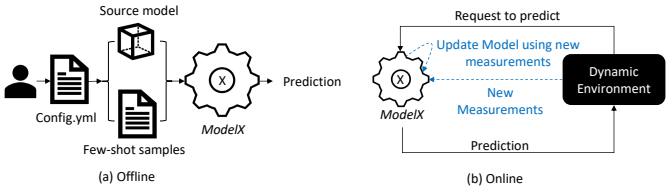


Fig. 3: How to use $\mathcal{M}odel\mathcal{X}$ in different scenarios.

III-F How to Use $\mathcal{M}odel\mathcal{X}$

Figure 3 presents an overview of how users can use $\mathcal{M}odel\mathcal{X}$ in their existing projects for both offline and online performance modeling scenarios. The workflow assumes users either have a pre-trained source model or a source dataset for building one. These options can be configured in the `config.yml` file, where users specify the path to the pre-trained model under `source_model_path` or the raw dataset under `source_data`. The `target_col` variable identifies which column is the performance metric to predict (Y). For test-time adaptation, the few-shot labeled samples can be provided in a file via the `few_shot_samples` variable. If `few_shot_samples` is missing, then the pipeline uses the source model directly for prediction. $\mathcal{M}odel\mathcal{X}$ automates all remaining steps, including data preprocessing, input alignment, and residual model construction.

Regardless of offline or online scenarios, users can build the source model offline. Sections V-A,V-B, and V-C evaluate $\mathcal{M}odel\mathcal{X}$ offline, while Section V-E emulates an online scenario of xApp knowledge transfer where a job scheduler refines its resource allocation-decision based on $\mathcal{M}odel\mathcal{X}$'s accurate predictions, enabling knowledge transfer without model retraining in dynamic environments. This work lays the foundation for handling performance modeling in dynamic environments in a data-efficient manner and removing the need for manual feature alignment or complete model retraining.

III-G Quantifying Difficulty of Knowledge Transfer for Explainability

To quantify the difficulty of knowledge transfer due to information mismatch, we propose a dissimilarity measure in Equation 9 considering only numerical features. Such a unified distance measure must handle (1) differences in feature names, numbers $(p \neq q)$, and orders, (2) variations in sample sizes $(N_s \neq N_t^L)$, and (3) numerical instability from high-dimensional feature spaces with limited samples.

As the **first step**, we use Principal Component Analysis (PCA) [17] to extract the top $d^* = \min(p, q, 10)$ principal components from each dataset X_s and X_t^L, preserving the most variation. This transformation produces $\hat{X}_s$ and $\hat{X}_s^L$, both with d^* dimensionality. **Next**, we address the sample size disparity between datasets when $N_s \neq N_t^L$. Since Grassmannian distance [18] calculation between two manifolds *requires* the dataset to have the same number of samples, we align their sample distributions using the Sinkhorn Optimal Transport [19] algorithm, implemented in the `Optimal Transport (OT)` Python library. The Sinkhorn algorithm

computes a probabilistic transport function, T, that partially assigns each source sample to multiple target samples with fractional weights. This algorithm ensures that datasets with different sample sizes can still be compared meaningfully without enforcing rigid one-to-one correspondences.

To achieve this, the algorithm constructs a cost matrix capturing pairwise distances between PCA-transformed samples and iteratively optimizes T. The resulting transport plan then calculates the eqivalent sample spaces: $\tilde{X}_s = T_m X_t^L, \quad \tilde{X}_t^L = T_m^T X_s$. Although we also experimented with random sampling as an alternative, our evaluation shows that without Sinkhorn transport, the manifold distance fails to capture a key property: the distance between a dataset and itself must be zero.

Finally, after ensuring alignment, we compute the Grassmannian manifold distance [18], which quantifies the structural similarity between datasets by measuring the alignment of their subspaces relative to a joint subspace. Initially, the algorithm identifies the principal directions of variability within each dataset (principal components) using the Singular Value Decomposition (SVD) method [20] such that $_, \sigma_s, _ = SVD(D_s)$ and $_, \sigma_t, _ = SVD(D_t)$. Then, the algorithm computes the principal components of their joint subspaces using $_, \sigma_j, _ = SVD([D_s : D_t])$. Next, the algorithm finds the alignment between $(D_s, [D_s : D_t])$ and $(D_t, [D_s : D_t])$ by calculating $_, \sigma_{sj}, _ = SVD(\sigma_s \cdot \sigma_j)$ and $_, \sigma_{tj}, _ = SVD(\sigma_t \cdot \sigma_j)$. The derived singular values, σ_{sj} and σ_{tj}, represent the cosine of the principal angles between the subspaces. Finally, the algorithm calculates $\theta_i^s = \cos^{-1}(\sigma_{sj}^i)$ and $\theta_i^t = \cos^{-1}(\sigma_{tj}^i)$ by taking the inverse cosine of the singular values along each principal component i, where $1 \leq i \leq d*$ and $d*$ is the number of principal components considered. Here, $0 < \theta < \frac{\pi}{2}$. These principal angles (θ^s and θ^t) provide a clear measure of how closely the subspaces align.

Number of principal components

$$severity = \frac{0.5}{d*} \sum_{i=1}^{d*} (\theta_i^s + \theta_i^t) \tag{9}$$

Principal angles

Equation 9 shows that $severity$ score is calculated by averaging the principal angles along all principal components. Since $d* \gg \frac{\pi}{2}$, Equation 9 results in $0 < severity < 1$.

IV. Experimental Setup

This section discusses the experimental setup common to the evaluation scenarios described in Section V.

IV-A Applications

Table I summarizes the datasets, including the number of features and samples, a subset of feature names, and the prediction target (bold). `CG`, `CoMD`, and `FT` are benchmarks from the NAS Parallel [21] suite. `CG` computes the conjugate gradient, widely used in spectral methods. Its irregular long-distance communication pattern makes it more communication-bound than compute-bound [21], [22]. `FT` solves a 3D partial differential equation using Fast Fourier Transforms (FFTs) and is also communication-bound. `LU` solves the compressible Navier-Stokes equations using a block lower and upper triangular system, representing a compute-bound, simplified computational fluid dynamics (CFD) application common in HPC.

`CoMD` is a force computation mini-application for molecular dynamics simulations, modeling material behavior at the atomic scale. It is typically memory-bound and compute-intensive due to irregular memory access patterns [23].

Kripke [24] is a proxy application that simulates 3D Sn deterministic particle transport and is compute-bound. `Kripke1` and `Kripke2` are datasets from two independent research teams that ran the same application on an Intel E5-2695v2 system but varied different parameters and measured different metrics, as shown in Table I.

`Hypre` [5] is a high-performance library for solving large, sparse linear systems in HPC. While some of the solvers implemented in `Hypre` can be compute bound, and others can be communication bound, depending on the problem size and hardware configuration.

`OpenMC` [25] is a production-level nuclear reactor simulation with millions of lines of code, using the Monte Carlo method for neutron transport. In contrast, `XSBench` [26] is a lightweight proxy that mimics `OpenMC`'s neutron transport kernel with far fewer lines of code. Both are compute-bound.

IV-B Preprocessing and Hyperparameter Tuning

We use min-max scaling for preprocessing numerical features and one-hot encoding for categorical ones. To identify the optimal hyperparameters for the NN used in f_s, $M_{\mathcal{A}}$, and $M_{\mathcal{R}}$, we use an automated hyperparameter tuning framework named Optuna [15] with the following parameter ranges: #-hidden layers $[3, 100]$, learning rate $[1e^{-5}, 1e^{-1}]$, #-neurons $[50, 150]$, with $dropout = 0.8$, $optimizer = Adam$, and $activation - function = RELU$.

IV-C Source Model Architectures

When available, we build f_s using multiple homogeneous datasets. However, the source datasets did not align in several evaluation scenarios, so we built separate single-application f_s. Since constructing f_s from heterogeneous datasets is an open and orthogonal research challenge, we leave it for future work. Instead, for a fair comparison, we use the same f_s for all test-time adaptation methods to isolate their individual benefits.

This work trains f_s using a fully connected neural network with batch normalization. The architecture varies by dataset: (1) single-application source models in Section V-A use 4 hidden layers with 100 neurons per layer, (2) multi-application source models in Section V-B use 4 hidden layers with 50 neurons per layer, (3) single-application models in Section V-B trained on Intel E5-2695v2 data use 6 hidden layers with 100 neurons per layer, and (4) ML benchmark applications use 3–6 hidden layers with 100–128 neurons per layer. For

all experiments, we maintain the same architecture for input alignment ($M_{\mathcal{A}}$) and residual ($M_{\mathcal{R}}$) models; their specific architectures vary across datasets.

IV-D Testing

For test-time adaptation experiments, we split the target dataset 90-10%, reserving 90% for testing. The same test split is used across all evaluations. To analyze sensitivity to sample size, we vary the number of few-shot samples from 1% to 5%, randomly selected from the 10% split. These same few-shot samples are used for test-time adaptation across all methods. All experiments follow 5-fold cross-validation, where we report the average Mean Square Error (MSE) across these five folds and the standard deviation when appropriate.

IV-E Baselines

For homogeneous knowledge transfer scenarios, we compare $\mathcal{M}odel\mathcal{X}$ with all of the baselines described in Section II: source only (f_s), linear probing, fine-tuning, few-shot retraining, and the supervised model, which represents the best case with up to 100× more target samples than test-time adaptation methods. For heterogeneous knowledge transfer scenarios, we compare $\mathcal{M}odel\mathcal{X}$ against the only viable alternative–training a model from scratch using the few-shot target samples.

V. Evaluations

This section tests the hypothesis that $\mathcal{M}odel\mathcal{X}$ enables knowledge transfer across heterogeneous datasets using a small number of labeled target samples, thereby being data-efficient. We evaluate $\mathcal{M}odel\mathcal{X}$ across five scenarios:

1) Evaluate transfer learning across architectures.
2) Evaluate transfer learning across applications.
3) Evaluate on ML benchmarks.
4) Micro-benchmark experiments.
5) Demonstrate impact on HPC job scheduling.

V-A Evaluate Transfer Learning Across Architectures

This experiment evaluates the effectiveness of $\mathcal{M}odel\mathcal{X}$ in cross-architecture knowledge transfer for HPC applications. The research question that drives this evaluation is: "Can knowledge from an application running on one system be transferred to predict its performance on a different system?" This scenario is particularly relevant in HPC procurement.

To study this question, we train a source model, f_s, for each application on an architecture and then use $\mathcal{M}odel\mathcal{X}$ to generalize its prediction for the same application on another architecture. Due to dataset heterogeneity, none of the baselines can be applied. Consequently, we compare $\mathcal{M}odel\mathcal{X}$ against the only viable alternative–training a new model from scratch using the few-shot target samples.

Dataset Used This experiment uses hardware performance counter data from OpenMC and XSBench run on two architectures: SB and BGQ. Table I illustrates feature mismatches across architectures. SB has 121 performance counters, while BGQ has 145. These datasets have similar features,

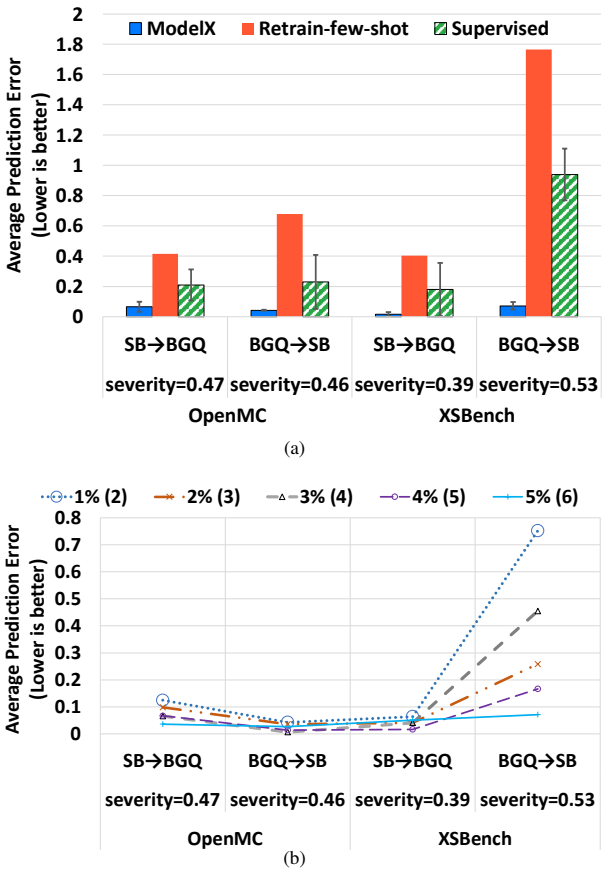


Fig. 4: (a) Average MSE of $\mathcal{M}odel\mathcal{X}$ during cross-architecture (xArch) knowledge transfer. $\mathcal{M}odel\mathcal{X}$ generalizes in all cases using only 6 samples—just 5% of the data needed to build a fully supervised target baseline (*supervised*), representing the best-case model. The supervised model for XSBench and OpenMC (separately) requires 26× more data. Despite minimal data, $\mathcal{M}odel\mathcal{X}$ remains robust, with low error variability across 5 trials, unlike retrain-with-few-shot, which showed excessive variance (not shown). This result is significant because $\mathcal{M}odel\mathcal{X}$ cuts data collection time from days to seconds. (b) As `severity` increases, $\mathcal{M}odel\mathcal{X}$ needs more samples (up to 6) to achieve very low prediction error consistently, demonstrating robustness.

such as PAPI_STL_ICY and PAPI_BR_INS as well as features that are unique to each architecture. For instance, BGQ dataset contains counters from AXU (specialized floating point unit on BGQ), XU (specialized integer unit on BGQ), L1P, and STL groups whereas SB dataset contains counters from MEM, DTLB, $L_{1,2,3}$ (last-level cache on SB). Since the number of hardware counters is too extensive, in Table I, we only present a subset of the feature names using regular expressions.

Existing work in the literature [6], [27] uses the parallel efficiency loss in these datasets as the target metric to predict the strong scaling characteristics of OpenMC and XSBench. We follow the same approach. Equation 10 defines the parallel

efficiency loss of an application:

$$\text{efficiency_loss}(p) = 1 - \frac{T(1)}{T(p) \times p} \tag{10}$$

Here, p represents the number of threads, $T(p)$ is the execution time with p threads, and $\frac{T(1)}{T(p)\times p}$ captures per-thread efficiency. This ratio-based metric isolates hardware effects by canceling software-related factors.

Observations From Figure 4(a), we observe that $\mathcal{M}odel\mathcal{X}$ consistently achieves lower prediction errors compared to few-shot retraining across all heterogeneous knowledge transfer scenarios. The effectiveness of $\mathcal{M}odel\mathcal{X}$ is really significant when the `severity` value, which quantifies the difficulty of knowledge transfer, is high. For instance, XSBench$_{BGQ\rightarrow SB}$, where the severity is 0.53, $\mathcal{M}odel\mathcal{X}$ achieves the lowest prediction error among the three alternatives.

When the distribution shift is less severe, as in OpenMC$_{SB\rightarrow BGQ}$, OpenMC$_{BGQ\rightarrow SQ}$, and XSBench$_{SB\rightarrow BGQ}$, $\mathcal{M}odel\mathcal{X}$ still outperforms retraining and the supervised model but achieves a smaller relative improvement. The supervised model trained on 160 samples–$26\times$ more than the 5% few-shot samples–fails to generalize, likely due to overfitting and the curse of dimensionality, as BGQ and SB have a large number of features, a common challenge in small-data regimes [28]. In contrast, the input alignment step of $\mathcal{M}odel\mathcal{X}$ automatically aligns source and target features, reducing distribution shifts to keep pre-trained embeddings of f_s relevant. The residual model then learns the discrepancy between $\text{M}_{\mathcal{A}}$'s predictions and the true labels Y_t^L, effectively performing error correction. This residual learning approach using model stacking [29] has been shown [30], [31] to be particularly effective in low-data scenarios, as it retains the strong baseline of f_s while making targeted adjustments with the few-shot data.

This result is particularly significant for OpenMC, a large-scale nuclear reactor simulation code, where extensive data collection would require substantial computational resources and time. With just 4 new samples (3% few-shot samples compared to the supervised model) from OpenMC, $\mathcal{M}odel\mathcal{X}$ enables accurate performance prediction from both Intel Sandy Bridge to IBM BG/Q and vice versa, minimizing the need for costly full-scale data collection.

V-B Evaluate Transfer Learning Across Applications

This experiment evaluates the effectiveness of $\mathcal{M}odel\mathcal{X}$ in xApp scenario for HPC applications. The research question driving this evaluation is: "Can knowledge from multiple applications on a given system be used to predict the performance of a new application on the same system?"

The assumption is that there will be some transferability between the source and target applications. This transferability arises from shared high-level features that influence performance across a class of applications, despite differences in specific details. For instance, a source model built from several applications with both compute and communication bounded kernels can learn that decreasing power cap affects compute bound kernels, whereas communication-bound applications are less affected; network bandwidth influences communication-bound kernels but not the compute-bound ones; compute-bound applications benefit from more threads up to the point of saturation, while communication-bound applications may experience diminishing returns due to synchronization and data transfer overheads; and finally, the number of threads and tasks affect the scalability behavior of an application, which in turn affects its execution time.

Dataset Used This experiment evaluates different test-time-adaptation techniques using both multi-application and single-application f_s. Table II presents the source and target datasets, % of few-shot samples used, and the results from various test-time adaptation methods. Table I describes a subset of the features used by the applications listed on Table II.

For XSBench and OpenMC, we use hardware performance counters as features and parallel efficiency loss as the target, as detailed in Section V-A. For all other applications, the target variable is execution time. These choices align with prior studies using these datasets. However, users can customize features and targets based on their specific needs.

Observations From Table II, we observe that $\mathcal{M}odel\mathcal{X}$ consistently outperforms few-shot retraining, source-only (0-shot), linear probing, and fine-tuning across both homogeneous and heterogeneous knowledge transfer scenarios, often requiring as few as $N_t^L = 2$ (1%) target-domain samples. In contrast, for xArch scenario, $\mathcal{M}odel\mathcal{X}$ requires up to $N_t^L = 6$ (5%) few-shot samples for robust generalization, which indicates that xArch knowledge transfer is likely more difficulty. When the difficulty in knowledge transfer is low (`severity` < 0.6 in all homogeneous knowledge transfer scenarios), the %-reduction in prediction error for $\mathcal{M}odel\mathcal{X}$ is lower than when difficulty in knowledge transfer increases (`severity` ≥ 0.6 for all heterogeneous knowledge transfer scenarios). The threshold is empirically observed. Further investigation shows that for scenarios with `severity` < 0.6, input alignment alone is sufficient; the residual model does not reduce error any further. These results indicate that even with identical features in homogeneous scenarios, $\mathcal{M}odel\mathcal{X}$'s input alignment mitigates distribution shifts, which makes the source model's learned feature embeddings effective.

In heterogeneous scenarios, $\mathcal{M}odel\mathcal{X}$ is the only approach capable of generalizing the source model, f_s. When feature heterogeneity makes knowledge transfer particularly difficult (`severity` ≥ 0.6), $\mathcal{M}odel\mathcal{X}$ significantly reduces prediction error compared to few-shot retraining. This is because few-shot samples alone are insufficient to train a model from scratch. We also observe that the residual model was necessary to achieve this reduction, input alignment is not sufficient.

To evaluate the effectiveness of $\mathcal{M}odel\mathcal{X}$, we compare its performance against a fully supervised model trained with up

TABLE II: Average MSE in cross-application (xApp) knowledge transfer. p and q denote the number of features. $\mathcal{M}odel\mathcal{X}$ outperforms all methods using only $N_t^L = 1\%$ samples in both homogeneous and heterogeneous transfer. The %-Improvement column shows error reduction over the best SOTA method. Input alignment suffices for low-to-moderate severity (`severity` < 0.6), while the residual model boosts accuracy in high-severity cases (`severity` ≥ 0.6). $\mathcal{M}odel\mathcal{X}$ surpasses the accuracy of supervised models trained on up to $100\times$ more data using just 2% few-shot samples (not shown).

Arch.	Source (p)	Target (q)	N_t^L (1%)	Few-shot retrain	f_s or 0-shot	Linear Probing	Fine-tuning	ModelX	%-Improvement	Severity
Homogeneous Knowledge Transfer										
Intel E5-2695v2	One-out experiment with `Kripke1`, LU, FT, CoMD, and CG (5)		7	0.361	0.054	0.399	0.391	**0.038**	29%	0.584
Intel Sandy Bridge (SB)	XSBench (121)	OpenMC (121)	2	1.686	1.627	1.365	1.183	**0.745**	37%	0.30
	OpenMC (121)	XSBench (121)	2	0.869	0.109	0.74	2.103	**0.026**	76%	0.30
IBM BG/Q (BGQ)	XSBench (145)	OpenMC (145)	2	2.297	1.39	1.11	1.571	**0.721**	35%	0.5
	OpenMC (145)	XSBench (145)	2	0.933	0.054	0.275	0.365	**0.012**	77%	0.43
Heterogeneous Knowledge Transfer										
Intel E5-2695v2	CG, LU, FT, CoMD (7)	`Kripke2` (23)	174	1.555	State-of-the-art methods fail due to feature mismatches			**0.072**	95%	0.60
		`Hypre` (21)	504	1.287				**0.184**	85%	0.66
	`Kripke2` (23)	`Hypre` (21)	504	1.155				**0.0009**	99%	0.70
	`Hypre` (21)	`Kripke2` (23)	174	0.663				**0.0004**	99%	0.70
	`Hypre` (21)	`Kripke1` (5)	5	0.363				**0.00002**	99%	0.70
	`Kripke1` (5)	`Hypre` (21)	504	22.358				**3.456**	84%	0.70

to $100\times$ more data, including a dataset with $50K$ samples. Remarkably, $\mathcal{M}odel\mathcal{X}$ matches their performance using at most 3% of the data across all experiments.

As in Section V-A, the input alignment step contributes by minimizing the distribution shifts between the source and target features. However, for heterogeneous datasets, it is not sufficient. This is where the residual model learns from both the labeled samples and the differences between the $M_{\mathcal{A}}$ model's predictions and the true values, to augment its knowledge. This residual learning approach is particularly effective in low-data scenarios, as it allows $\mathcal{M}odel\mathcal{X}$ to leverage pre-existing knowledge to generalize.

V-C Evaluate $\mathcal{M}odel\mathcal{X}$ using ML Benchmarks

This experiment evaluates the effectiveness of $\mathcal{M}odel\mathcal{X}$ in transferring knowledge across ML datasets where the source and target partitions have identical features but may differ in data distributions (homogeneous scenario with distribution shifts). The key research question is: "Can we adapt a model trained on one subset of a dataset to another subset with the same features but potentially different data distributions?" In this experiment, we compare our approach to existing transfer learning methods. While prior studies [32] have used these datasets for out-of-distribution (OOD) generalization, the extent of distribution shifts across partitions has been subjective. This work systematically quantifies the shift using `severity` and connects it to the necessity of test-time adaptation.

Dataset Used We evaluate four widely used OOD ML datasets: `Airfoil`, `NO2`, `Crime`, and `SkillCraft`, each with multiple partitions enabling cross-domain knowledge transfer. Table I lists key features and highlights the prediction target in bold. Following [32], we split each dataset into training and test partitions, designating one as the source domain and another as the target. Table III presents transfer scenarios in the format source partition $\rightarrow$ target partitions (few-shot samples), alongside prediction errors and transfer difficulty (`severity`). We compare $\mathcal{M}odel\mathcal{X}$ against four baselines—LP, FT, f_s, and retraining with 1%-5% few-shot samples, and only show the results for 1% few-shot samples.

TABLE III: Normalized average MSE across all ML benchmarks: `Airfoil`, `NO2`, `Crime`, and `SkillCraft`. `Airfoil` and `NO2` require no adaptation as they exhibit no distribution shift. As severity increases due to distribution shifts between source and target partitions of `Crime` and `SkillCraft`, $\mathcal{M}odel\mathcal{X}$ outperforms all existing methods in knowledge transfer, particularly because the input alignment phase enables $\mathcal{M}odel\mathcal{X}$ to effectively utilize the source model's knowledge.

Src $\rightarrow$ Target (N_t^L)	Models					Severity
	Few-shot retrain	f_s	LP	FT	ModelX	
`Airfoil` $S_1 \rightarrow S_{2-4}$ (3)	1.68E-7	1.18E-9	2.71E-1	2.69E-1	1.85E-8	1e-6
`NO2` $S_1 \rightarrow S_{2-4}$ (1)	0.113	0.001	0.019	0.0217	0.016	1e-6
`Crime` $S_9 \rightarrow S_{6,12,25}$ (1)	0.600	0.018	0.080	0.087	0.0174	0.089
`SkillCraft` $L_1 \rightarrow L_{3-6}$ (6)	0.434	0.141	0.121	0.467	0.060	0.38

Observations From Table III, we can observe that when datasets have no distribution shift (`severity` $\simeq 0$), for instance, in `Airfoil` and `NO2`, the source model suffices. In this case, the test-time adaptation can cause overfitting, thus reducing the benefit of generalization. Otherwise, when `severity` > 0, similar to the HPC scenarios, $\mathcal{M}odel\mathcal{X}$ outperforms all other SOTA methods. Specifically, $\mathcal{M}odel\mathcal{X}$

outperforms linear probing and fine-tuning because both of these SOTA methods modify f_s whereas $\mathcal{M}odel\mathcal{X}$ augments the source model's knowledge using the new information learned from the target few-shot samples.

V-D Micro-benchmark Experiments

Ablation Study To identify when input alignment ($M_{\mathcal{A}}$) is sufficient and when residual model is necessary ($M_{\mathcal{R}}$), we conduct an ablation study. Our experiments reveal that when severity is below 0.6, $M_{\mathcal{A}}$ alone is sufficient to achieve either comparable or less prediction error compared to the best-case supervised model. However, when severity is 0.6 or higher, $M_{\mathcal{R}}$ becomes essential to capture residual relationships not addressed by input alignment alone. This pattern is consistent across all but two datasets: (1) `Hypre` $\rightarrow$ `Kripke2`, where $M_{\mathcal{R}}$ surpasses the best-case model when using 3% few-shot samples. (2) `Hypre` $\rightarrow$ `Kripke1`, where more few-shot samples did not improve $M_{\mathcal{R}}$'s accuracy. The low prediction error suggests input alignment effectively projects the target space for source model generalization, making the residual model unnecessary.

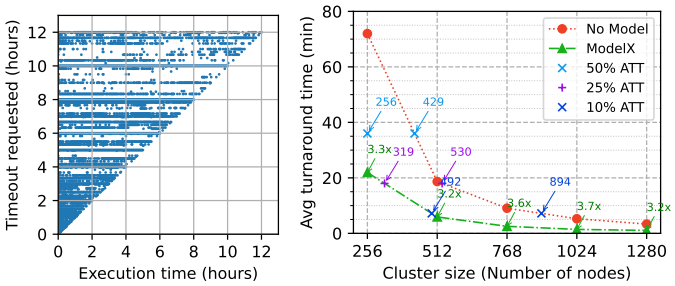
Sensitivity Analysis We evaluate the impact of few-shot samples on accuracy through a sensitivity study spanning 800 experiments, testing 1% to 5% of the target data. In most cases, $\mathcal{M}odel\mathcal{X}$ outperforms a best-case supervised model trained with up to $100\times$ more data, even with just 1–3% of the target samples. When `severity` > 0.6, adding more few-shot samples generally improves stacked model accuracy, though effects vary by dataset. Figure 4(b) highlights the challenge of heterogeneous xArch transfer, where $\mathcal{M}odel\mathcal{X}$ requires up to 5% few-shot samples for stable performance.

Cost of Training and Inference The average training time for a source model with 6 hidden layers and 100 neurons per layer is $140.58s$ for the largest dataset in this study ($50,396$ samples, 21 features). For 1% few-shot samples (174 target samples), the average test-time adaptation takes $45.83s$, with an average inference time of $0.78s$ per query across five trials.

V-E Demonstrate the Impact of $\mathcal{M}odel\mathcal{X}$ on HPC Job Scheduling

In this section, we discuss the real-world applicability of $\mathcal{M}odel\mathcal{X}$ in the context of HPC job scheduling, emulating the online modeling scenario shown in Figure 3b. Specifically, we investigate whether replacing job configurations (e.g., number of nodes, network placement, power cap), often chosen by users ad hoc, with alternatives recommended by $\mathcal{M}odel\mathcal{X}$, reduces job turnaround time. For practical deployment, users could provide a plugin filter that exposes runnable configurations as valid candidates. This experiment only considers previously seen configurations.

Dataset Used To evaluate the impact of model-guided scheduling, we generate synthetic job streams of applications from the xApp experiment in Section V-B. To ensure the job statistics remain consistent with the real-world HPC workload patterns observed in production clusters, we use the exact



(a) (b)

Fig. 5: (a) User-specified job timeouts vs. actual execution time [33], [34]. (b) The $\mathcal{M}odel\mathcal{X}$-guided scheduling reduces ATT by 71% and accelerates by a factor of 3.4x on average over clusters of various sizes. The base ATT is 1hr 12 mins long with 256 nodes. The plot shows the size of HPC cluster required to achieve 50%, 25% and 10% of that with and without $\mathcal{M}odel\mathcal{X}$. $\mathcal{M}odel\mathcal{X}$-guided scheduling requires 42% less nodes than the baseline to achieve the same level of ATT.

job submission times as well as the statistics of timeout parameter, shown in Figure 5(a), from a publicly available job trace [33], [34] recorded by a Cluster Management System (CMS) between August 22, 2018, and November 18, 2020.

Experimental Setup Our ML-based scheduling simulator[1], implemented in Python 3.9.12, Pandas 2.2.2, NumPy 1.26.4, follows a three-step process.

Step A: job stream generation We randomly sample N jobs from M execution samples in the xApp dataset and assign timeout values based on the real-world distributions in Figure 5(a) corresponding to the execution time. We choose N to be 70,000 to represent jobs of a week. The submission times of these jobs match those from a randomly chosen week in the log. This creates a job stream for scheduling.

Step B: job configuration selection The scheduler uses $\mathcal{M}odel\mathcal{X}$, discussed in Section V-B, to predict if a job can maintain similar performance while using fewer nodes. This decision process leverages knowledge learned from prior jobs, and aligns new job configurations to ensure accurate predictions despite distribution shifts. The scheduler only selects a new configuration if its predicted runtime is within λ (0–10%) of the user-requested configuration. We find no additional benefit beyond $\lambda = 10\%$. With this transformation, the increased runtime may result in job expiration before completion. Such a job is resubmitted with an increased timeout value. However, we see no or few such cases with $\lambda = 10\%$ and observed no noticeable impact on performance.

Step C: scheduling and evaluation We apply the First Come, First Serve (FCFS) policy with EASY backfilling [9]. Note that the scheduling decisions, highlighted in this experiment, are made at the downstream of the accounting and priority management of the HPC scheduler. The FCFS policy may lead to inefficiencies if large jobs block smaller ones. The EASY

[1]The link to the software repository will be provided once accepted.

backfilling mitigates this by allowing smaller jobs to move ahead in the queue, provided they do not delay the earliest-starting job. This improves resource utilization.

To evaluate scheduling efficiency, we use the metric, average turnaround time (ATT), which measures the time from job submission to completion. By integrating $\mathcal{M}odel\mathcal{X}$ for job configuration selection, we observe how ATT improves compared to default user-requested configurations, while also evaluating the impact of varying cluster sizes.

Observations Figure 5(b) shows that replacing the user-defined node count with the model-recommended values reduces ATT by 71% (or accelerates by a factor of 3.4x) on average over clusters of various sizes when the scheduler allows up to 10% slowdown. This allowance enables the scheduler to optimize node allocation to improve the overall cluster efficiency by backfilling more jobs. Although individual job performance may degrade by up to 10% the cluster-wide productivity boost outweighs this tradeoff. Beyond this threshold, we observed no noticeable ATT improvement. In Figure 5(b), the base ATT is 1hr 12 mins long with 256 nodes. The plot shows the size of HPC cluster required to achieve 50%, 25% and 10% of that with and without $\mathcal{M}odel\mathcal{X}$. The $\mathcal{M}odel\mathcal{X}$-guided scheduling requires 42% fewer nodes on average than the baseline to achieve the same level of ATT. Such insights can enable informed procurement decisions.

While an in-depth study of more optimized and sophisticated scheduling methods is beyond the scope of this work, these results clearly showcase the applicability of automated decision-making in HPC scheduling. This highlights the need for a data-driven approach, augmenting a heuristic-based one that can provide more accurate runtime estimates and guide users and schedulers to make better allocation and scheduling decisions.

$\mathcal{M}odel\mathcal{X}$ lays out a foundation for online adaptation of intelligent systems via the technique to expand existing knowledge with very few new observations. The performance model we use itself needs to be generalizable to fully leverage this framework. Therefore, we rely on performance counters as features rather than information that is not directly relevant to how an application behaves on particular hardware [10], [35]. This, however, makes the utilization of the model somewhat challenging as the counters are only collected in-situ. The most practical use cases include ensemble workflows or batches of ML interferences where there is a large collection of similarly performing workloads. In such a case, the first few jobs can be profiled to collect necessary features, and represent the rest of the collection by the features. Only when a prediction turns out to be significantly inaccurate, the particular job can be rerun for profiling. There exist works that do not depend on counter features while complementing this approach [36]. However, the prediction based on static analysis fails to capture the performance variability with the application input [37].

VI. Related Work

Transfer learning has been widely studied in fields such as NLP for text sentiment classification [38], text classification [2], software engineering [39] but more heavily in the visual domain including image classification [40]–[45], human activity classification [4], [46]–[48] Semi-supervised transfer learning with limited labeled target data has also been used in the visual domains [49]–[52]. Yu et al. [53] propose fine-tuning a pre-trained classifier with novel classes from the target domain. Abuduweili et al. [54] propose an adaptive technique by fine-tuning Resnet 50 [55] model on CIFAR 10 [56] data. Choi et al. [3] propose using activations of feature maps from multiple layers to transfer knowledge [57] in music-related regression tasks, which are domain-specific. However, these image or domain-specific classification techniques do not apply to regression settings. David et al. [58] extend two classification-based transfer learning techniques into regression settings but do not handle heterogeneous domains. Jochen et al. [59] propose a prediction function re-weighting strategies for addressing distribution shifts but do not quantify them.

Performance modeling of applications using machine learning is a well-studied topic in HPC [10], [60]–[65]. The cost and overhead of data collection and the unavailability of labeled data make transfer learning with few-shot samples a popular approach [5], [66]–[69]. [5] uses a subspace-based learning approach for transfer learning but do not address distribution shift or heterogeneous domains. Tong et al. [67] discuss the distribution dissimilarity between source and target domains but do not handle domain heterogeneity, Madireddy et al [68] work on a similar problem of identifying "concept drift" in applications using an online Bayesian change point detection algorithm and propose a transformation to realign training data before modeling, but their work does not address transferring knowledge across heterogeneous domains.

VII. Conclusions

This work proposes a novel few-shot-learning-based method called $\mathcal{M}odel\mathcal{X}$ for test-time adaptation. It enables data-efficient knowledge transfer across homogeneous and heterogeneous datasets by addressing distribution shifts in feature values and structural mismatches. $\mathcal{M}odel\mathcal{X}$ outperforms traditional transfer learning methods and matches or exceeds fully supervised models trained on $100\times$ more data, using just 1-5% of the samples. We systematically quantify transfer difficulty, identifying when input alignment suffices and when a residual model is needed. Finally, we demonstrate $\mathcal{M}odel\mathcal{X}$'s impact by showing how it improves HPC job scheduling, reducing application turnaround time by 71%.

Acknowledgment

This material is based upon work supported by the U.S. Department of Energy, Office of Science under Award Number DE-SC0022843. This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344.

LLNL-CONF-870385. This work was supported in part by LLNL LDRD projects 23-ERD-045 and 24-SI-005. We also thank Texas Advanced Computing Center (TACC) The University of Texas at Austin.

REFERENCES

[1] T. Patki, J. J. Thiagarajan, A. Ayala, and T. Z. Islam, "Performance optimality or reproducibility: that is the question," in *International Conference for High Performance Computing, Networking, Storage and Analysis*, 2019, pp. 1–30.

[2] C. B. Do and A. Y. Ng, "Transfer learning for text classification," *Advances in neural information processing systems*, vol. 18, 2005.

[3] K. Choi, G. Fazekas, M. Sandler, and K. Cho, "Transfer learning for music classification and regression tasks," 2017.

[4] D. Cook, K. D. Feuz, and N. C. Krishnan, "Transfer learning for activity recognition: A survey," *Knowledge and information systems*, vol. 36, pp. 537–556, 2013.

[5] A. Marathe, R. Anirudh, N. Jain, A. Bhatele, J. Thiagarajan, B. Kailkhura, J.-S. Yeom, B. Rountree, and T. Gamblin, "Performance modeling under resource constraints using deep transfer learning," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2017, pp. 1–12.

[6] J. J. Thiagarajan, R. Anirudh, B. Kailkhura, N. Jain, T. Islam, A. Bhatele, J. Yeom, and T. Gamblin, "Paddle: Performance analysis using a data-driven learning environment," in *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May, pp. 784–793.

[7] A. Kumar, A. Raghunathan, R. Jones, T. Ma, and P. Liang, "Fine-tuning can distort pretrained features and underperform out-of-distribution," 2022.

[8] M. A. Bender, B. C. Kuszmaul, and W. Kuszmaul, "Linear probing revisited: Tombstones mark the demise of primary clustering," in *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, 2022, pp. 1171–1182.

[9] J. Lelong, V. Reis, and D. Trystram, "Tuning easy-backfilling queues," in *Job Scheduling Strategies for Parallel Processing - 21st International Workshop, JSSPP 2017, Orlando, FL, USA, June 2, 2017, Revised Selected Papers*, ser. Lecture Notes in Computer Science, D. Klusácek, W. Cirne, and N. Desai, Eds., vol. 10773. Springer, 2017, pp. 43–61.

[10] D. Nichols, A. Movsesyan, J. Yeom, A. Sarkar, D. Milroy, T. Patki, and A. Bhatele, "Predicting cross-architecture performance of parallel programs," in *Proceedings of the IEEE International Parallel Distributed Processing Symposium*, ser. IPDPS '24. IEEE Computer Society, may 2024.

[11] J. Liang, R. He, and T. Tan, "A comprehensive survey on test-time adaptation under distribution shifts," 2023.

[12] A. Ramdan, A. Heryana, A. Arisal, R. B. S. Kusumo, and H. F. Pardede, "Transfer learning and fine-tuning for deep learning-based tea diseases detection on small datasets," in *2020 International Conference on Radar, Antenna, Microwave, Electronics, and Telecommunications (ICRAMET)*, 2020, pp. 206–211.

[13] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2016.

[14] R. Sennrich and B. Haddow, "Linguistic input features improve neural machine translation," *arXiv preprint arXiv:1606.02892*, 2016.

[15] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2623–2631.

[16] D. H. Wolpert, "Stacked generalization," *Neural networks*, vol. 5, no. 2, pp. 241–259, 1992.

[17] M. F. I. Ibrahim and A. A. Al-Jumaily, "Pca indexing based feature learning and feature selection," in *2016 8th Cairo International Biomedical Engineering Conference (CIBEC)*, 2016, pp. 68–71.

[18] Z. Huang, J. Wu, and L. Van Gool, "Building deep networks on grassmann manifolds," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[19] K. Pham, K. Le, N. Ho, T. Pham, and H. Bui, "On unbalanced optimal transport: An analysis of sinkhorn algorithm," in *International Conference on Machine Learning*. PMLR, 2020, pp. 7673–7682.

[20] M. E. Wall, A. Rechtsteiner, and L. M. Rocha, "Singular value decomposition and principal component analysis," in *A practical approach to microarray data analysis*. Springer, 2003, pp. 91–109.

[21] "NASA Advanced Supercomputing Division, NAS Parallel Benchmark Suite v3.3."

[22] S. Saini, J. Chang, R. Hood, and H. Jin, "A scalability study of columbia using the nas parallel benchmarks," *Journal of Comput. Methods in Sci. and Engr*, 2006.

[23] G. Lawson, V. Sundriyal, M. Sosonkina, and Y. Shen, "Experimentation procedure for offloaded mini-apps executed on cluster architectures with xeon phi accelerators," 2015. [Online]. Available: https://arxiv.org/abs/1509.02135

[24] A. J. Kunen, T. S. Bailey, and P. N. Brown, "Kripke - a massively parallel transport mini-app."

[25] P. K. Romano and B. Forget, "The openmc monte carlo particle transport code," *Annals of Nuclear Energy*, vol. 51, pp. 274–281, 2013.

[26] J. Tramm, A. Siegel, T. Islam, and M. Schulz, "XSBench-the Development and Verification of a Performance Abstraction for Monte Carlo Reactor Analysis," *The Role of Reactor Physics toward a Sustainable Future (PHYSOR)*, 2014.

[27] T. Z. Islam, J. J. Thiagarajan, A. Bhatele, M. Schulz, and T. Gamblin, "A machine learning framework for performance coverage analysis of proxy applications," in *SC'16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2016, pp. 538–549.

[28] M. Belkin, D. Hsu, S. Ma, and S. Mandal, "Reconciling modern machine-learning practice and the classical bias–variance trade-off," *Proceedings of the National Academy of Sciences*, vol. 116, no. 32, p. 15849–15854, Jul. 2019. [Online]. Available: http://dx.doi.org/10.1073/pnas.1903070116

[29] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, P.-A. Manzagol, and L. Bottou, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion." *Journal of machine learning research*, vol. 11, no. 12, 2010.

[30] Z. Yang, Y. Yu, C. You, J. Steinhardt, and Y. Ma, "Rethinking bias-variance trade-off for generalization of neural networks," in *International Conference on Machine Learning*. PMLR, 2020, pp. 10 767–10 777.

[31] T. Wang, K. Zhang, J. Thé, and H. Yu, "Accurate prediction of band gap of materials using stacking machine learning model," *Computational Materials Science*, vol. 201, p. 110899, 2022.

[32] H. Yao, Y. Wang, L. Zhang, J. Zou, and C. Finn, "C-mixup: Improving generalization in regression," 2022.

[33] "LLNL/LAST: Livermore Archive for System Telemetry (LAST)." [Online]. Available: https://github.com/LLNL/LAST/tree/main

[34] T. Patki, A. Bertsch, I. Karlin, D. H. Ahn, B. Van Essen, B. Rountree, B. R. de Supinski, and N. Besaw, "Monitoring large scale supercomputers: A case study with the lassen supercomputer," in *2021 IEEE International Conference on Cluster Computing (CLUSTER)*, 2021, pp. 468–480.

[35] L. Li, T. Flynn, and A. Hoisie, "Learning generalizable program and architecture representations for performance modeling," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, Nov. 2024.

[36] D. Nichols, A. Marathe, H. Menon, T. Gamblin, and A. Bhatele, "Hpc-coder: Modeling parallel programs using large language models," in *ISC High Performance*, May 2024.

[37] J.-S. Yeom, J. J. Thiagarajan, A. Bhatele, G. Bronevetsky, and T. Kolev, "Data-driven performance modeling of linear solvers for sparse matrices," in *Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), International Workshop on*. IEEE, 2016, pp. 32–42.

[38] R. Liu, Y. Shi, C. Ji, and M. Jia, "A survey of sentiment analysis based on transfer learning," *IEEE access*, vol. 7, pp. 85 401–85 412, 2019.

[39] J. Chen, K. Hu, Y. Yu, Z. Chen, Q. Xuan, Y. Liu, and V. Filkov, "Software visualization and deep transfer learning for effective software defect prediction," in *Proceedings of the ACM/IEEE 42nd international conference on software engineering*, 2020, pp. 578–589.

[40] E. Deniz, A. Şengür, Z. Kadiroğlu, Y. Guo, V. Bajaj, and Ü. Budak, "Transfer learning based histopathologic image classification for breast cancer detection," *Health information science and systems*, vol. 6, pp. 1–7, 2018.

[41] S. Khan, N. Islam, Z. Jan, I. U. Din, and J. J. C. Rodrigues, "A novel deep learning based framework for the detection and classification of breast cancer using transfer learning," *Pattern Recognition Letters*, vol. 125, pp. 1–6, 2019.

[42] Y. Pathak, P. K. Shukla, A. Tiwari, S. Stalin, and S. Singh, "Deep transfer learning based classification model for covid-19 disease," *Irbm*, vol. 43, no. 2, pp. 87–92, 2022.

[43] M. Hussain, J. J. Bird, and D. R. Faria, "A study on cnn transfer learning for image classification," in *Advances in Computational Intelligence Systems: Contributions Presented at the 18th UK Workshop on Computational Intelligence, September 5-7, 2018, Nottingham, UK*. Springer, 2019, pp. 191–202.

[44] S. Coulibaly, B. Kamsu-Foguem, D. Kamissoko, and D. Traore, "Deep neural networks with transfer learning in millet crop images," *Computers in industry*, vol. 108, pp. 115–120, 2019.

[45] Y. Zhu, Y. Chen, Z. Lu, S. Pan, G.-R. Xue, Y. Yu, and Q. Yang, "Heterogeneous transfer learning for image classification," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 25, no. 1, 2011, pp. 1304–1309.

[46] J. Park, R. J. Javier, T. Moon, and Y. Kim, "Micro-doppler based classification of human aquatic activities via transfer learning of convolutional neural networks," *Sensors*, vol. 16, no. 12, p. 1990, 2016.

[47] A. Butt, S. Narejo, M. Anjum, M. Yonus, M. A. Memon, and A. Samejo, "Fall detection using lstm and transfer learning," *Wireless Personal Communications*, vol. 126, pp. 1–18, 06 2022.

[48] X. Cao, D. Wipf, F. Wen, G. Duan, and J. Sun, "A practical transfer learning algorithm for face verification," in *2013 IEEE International Conference on Computer Vision*, 2013, pp. 3208–3215.

[49] C. Bettini, G. Civitarese, and R. Presotto, "Personalized semi-supervised federated learning for human activity recognition," *arXiv preprint arXiv:2104.08094*, 2021.

[50] I. D. Apostolopoulos, N. D. Papathanasiou, and G. S. Panayiotakis, "Classification of lung nodule malignancy in computed tomography imaging utilising generative adversarial networks and semi-supervised transfer learning," *Biocybernetics and Biomedical Engineering*, vol. 41, no. 4, pp. 1243–1257, 2021.

[51] A. Saeed, T. Ozcelebi, and J. Lukkien, "Multi-task self-supervised learning for human activity detection," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 3, no. 2, pp. 1–30, 2019.

[52] V. Cheplygina, M. de Bruijne, and J. P. Pluim, "Not-so-supervised: a survey of semi-supervised, multi-instance, and transfer learning in medical image analysis," *Medical image analysis*, vol. 54, pp. 280–296, 2019.

[53] Z. Yu, L. Chen, Z. Cheng, and J. Luo, "Transmatch: A transfer-learning scheme for semi-supervised few-shot learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[54] A. Abuduweili, X. Li, H. Shi, C.-Z. Xu, and D. Dou, "Adaptive consistency regularization for semi-supervised transfer learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 6923–6932.

[55] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015.

[56] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images."

[57] M. Rohrbach, S. Ebert, and B. Schiele, "Transfer learning in a transductive setting," in *Advances in Neural Information Processing Systems*, C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger, Eds., vol. 26. Curran Associates, Inc., 2013.

[58] D. Pardoe and P. Stone, "Boosting for regression transfer," in *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ser. ICML'10. Madison, WI, USA: Omnipress, 2010, p. 863–870.

[59] S. Al-Stouhi and C. K. Reddy, "Adaptive boosting for transfer learning using dynamic updates," in *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2011, Athens, Greece, September 5-9, 2011. Proceedings, Part I 11*. Springer, 2011, pp. 60–75.

[60] N. Yigitbasi, T. Willke, G. Liao, and D. Epema, "Towards machine learning-based auto-tuning of mapreduce," 08 2013, pp. 11–20.

[61] N. Siegmund, S. Kolesnikov, C. Kästner, S. Apel, D. Batory, M. Rosenmüller, and G. Saake, "Predicting performance via automated feature-interaction detection," 05 2012.

[62] Y. Zhang, J. Guo, E. Blais, and K. Czarnecki, "Performance prediction of configurable software systems by fourier learning (t)," 2015, pp. 365–373.

[63] F. Wu, W. Weimer, M. Harman, Y. Jia, and J. Krinke, "Deep parameter optimisation," in *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*, ser. GECCO '15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: https://doi.org/10.1145/2739480.2754648

[64] N. Siegmund, A. Grebhahn, S. Apel, and C. Kästner, "Performance-influence models for highly configurable systems," 2015.

[65] A. Sarkar, J. Guo, N. Siegmund, S. Apel, and K. Czarnecki, "Cost-efficient sampling for performance prediction of configurable systems (t)," 11 2015, pp. 342–352.

[66] M. A. Javidian, P. Jamshidi, and M. Valtorta, "Transfer learning for performance modeling of configurable systems: A causal analysis," *arXiv preprint arXiv:1902.10119*, 2019.

[67] T. Liu, S. Alibhai, J. Wang, Q. Liu, X. He, and C. Wu, "Exploring transfer learning to reduce training overhead of hpc data in machine learning," in *2019 IEEE International Conference on Networking, Architecture and Storage (NAS)*. IEEE, 2019, pp. 1–7.

[68] S. Madireddy, P. Balaprakash, P. Carns, R. Latham, G. K. Lockwood, R. Ross, S. Snyder, and S. M. Wild, "Adaptive learning for concept drift in application performance modeling," in *Proceedings of the 48th International Conference on Parallel Processing*, 2019, pp. 1–11.

[69] P. Malakar, P. Balaprakash, V. Vishwanath, V. Morozov, and K. Kumaran, "Benchmarking machine learning methods for performance modeling of scientific applications," in *2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*. IEEE, 2018, pp. 33–44.